

Kallang Freight - etcd Disaster Recovery Runbook (TrackLane production)

Document class: Platform Engineering / DR Owner: TrackLane Platform Team Applies to: kubeadm-managed control planes only Revision: 2026-09-05

❑ DO NOT RUN ANY COMMAND IN SECTION 3 ON A SHARED OR CLASSROOM CLUSTER

Section 3 is a destructive procedure. It stops the API server, replaces the etcd data directory and rewrites cluster state. Executing it against the KCNA training cluster will break the cluster for every other learner in the room and will invalidate every other lab in this course.

In this course this runbook is a reading exercise. You will mount it into a Pod and read it. You will not execute it. Sections 1 and 2 (snapshot and verify) are non-destructive and *are* performed in Lab 20.

Practise Section 3 only on a cluster you personally created and are willing to destroy - for example a throwaway single-node kind cluster of your own, on your own machine, with no one else connected.

1. Take a snapshot (NON-DESTRUCTIVE - performed in Lab 20)

etcdctl snapshot save opens a normal client connection to a *running* etcd, streams a point-in-time copy of the keyspace, and writes it to a file. It does not pause etcd, does not lock the keyspace and does not modify any data.

Required inputs, all read from the etcd static Pod manifest:

Input	kubeadm default
endpoint	https://127.0.0.1:2379
CA certificate	/etc/kubernetes/pki/etcd/ca.crt
client certificate	/etc/kubernetes/pki/etcd/server.crt
client key	/etc/kubernetes/pki/etcd/server.key

```
etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /var/lib/etcd/snapshot-$(date -u +%Y%m%dT%H%M%S).db
```

2. Verify the snapshot (NON-DESTRUCTIVE - performed in Lab 20)

A snapshot you have never verified is not a backup. snapshot status reads the file offline - it never contacts a server - and reports the hash, revision, total keys and file size.

```
etcdctl snapshot status /var/lib/etcd/snapshot-<stamp>.db -w table
```

Binary note. snapshot status and snapshot restore operate on a file, not a server, so etcd moved them out of etcdctl and into etcdutl. They

are deprecated in etcdctl from etcd v3.5 and **removed from etcdctl in etcd v3.6**. snapshot save remains an etcdctl command because it does talk to a running server. Check which binaries your etcd image ships before relying on either.

Acceptance criteria for a usable snapshot:

* TOTAL KEYS is in the expected order of magnitude for the cluster (a small training cluster is typically low hundreds to low thousands).

* SIZE is non-trivial (a few MB at minimum; a near-zero file is a failed save).

* The command exits 0 and prints a hash.

3. ☐ RESTORE - DESTRUCTIVE. DO NOT RUN ON A SHARED CLUSTER ☐

Read this section. Do not execute it here.

Preconditions before a real restore:

1. You have confirmed the cluster is genuinely unrecoverable, not merely degraded. Restoring loses every write made after the snapshot was taken.
2. You have a verified snapshot (Section 2) and know its wall-clock time.
3. You have a maintenance window and stakeholder sign-off - the API server is unavailable for the duration.
4. For a multi-member cluster: every member is restored from the same snapshot, with matching --initial-cluster topology. Restoring one member from a snapshot while others keep their old data corrupts the Raft group.

Outline of the procedure on a kubeadm single control-plane node:

```
# 3.1 Stop the control plane by moving its static Pod manifests aside.
#     The kubelet watches this directory and will tear the Pods down.
mv /etc/kubernetes/manifests/kube-apiserver.yaml /etc/kubernetes/manifests.bak/
mv /etc/kubernetes/manifests/etcd.yaml /etc/kubernetes/manifests.bak/
mv /etc/kubernetes/manifests/kube-controller-manager.yaml /etc/kubernetes/manifests.bak/
mv /etc/kubernetes/manifests/kube-scheduler.yaml /etc/kubernetes/manifests.bak/

# 3.2 Restore the snapshot into a NEW directory. Never restore over a live
#     data directory.
etcdctl snapshot restore /var/lib/etcd-backup/snapshot-<stamp>.db \
  --name=<member-name> \
  --initial-cluster=<member-name>=https://<node-ip>:2380 \
  --initial-cluster-token=etcd-cluster-restored \
  --initial-advertise-peer-urls=https://<node-ip>:2380 \
  --data-dir=/var/lib/etcd-restored

# 3.3 Point the etcd static Pod at the restored data directory by editing the
#     hostPath volume in etcd.yaml, or move the directory into place:
mv /var/lib/etcd /var/lib/etcd-preRestore
mv /var/lib/etcd-restored /var/lib/etcd

# 3.4 Put the static Pod manifests back. The kubelet recreates the Pods.
mv /etc/kubernetes/manifests.bak/*.yaml /etc/kubernetes/manifests/

# 3.5 Wait for the control plane, then verify.
kubectl get --raw='/readyz?verbose'
kubectl get nodes
kubectl get pods -A
```

What restore does NOT recover

- * Anything written after the snapshot timestamp. Your RPO is your snapshot interval, full stop.
- * PersistentVolume contents. etcd stores the PV and PVC *objects*, not the bytes on the disk. Volume data needs its own backup - a CSI VolumeSnapshot, a storage-array snapshot, or an application-level dump.
- * Secrets that were encrypted at rest, unless you also still hold the EncryptionConfiguration key material. Back up the encryption keys separately and never in the same blast radius as the snapshot.
- * Certificates in /etc/kubernetes/pki. Back these up alongside the snapshot.

4. Backup policy (what "good" looks like)

Control	TrackLane production target
Snapshot interval	every 30 minutes (RPO ≤ 30 min)
Restore drill	quarterly, on a rebuilt cluster, timed against the RTO
RTO target	60 minutes from decision to readyz ok
Retention	48 hourly, 14 daily, 12 monthly
Off-node copy	mandatory - a snapshot on the failed node is not a backup
Encryption	snapshots encrypted at rest with a key held outside the cluster
Verification	snapshot status on every snapshot; full restore drill quarterly
Also backed up	/etc/kubernetes/pki, EncryptionConfiguration, PV data

5. Escalation

If a restore is being considered, page the on-call platform lead before executing Section 3. A restore is a one-way door.